

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. L'usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On s'intéresse à des suites binaires, c'est-à-dire des chaînes ne contenant que les chiffres 0 et 1. Le **complément binaire** d'une suite est obtenu en remplaçant chaque 0 par 1, et chaque 1 par 0.

`'010011' → '101100'`

1. Écrire une fonction Python `complement_binaire` qui prend en entrée un chaîne de caractères composée de '0' et '1' et renvoie la chaîne correspondant à son complément binaire.

Exemples :

```
>>> complement_binaire("010011")
'101100'
>>> complement_binaire("1111")
'0000'
```

2. On construit la suite de Thue-Morse de manière itérative :

$$t_0 = '0', \quad t_{n+1} = t_n + \text{complement_binaire}(t_n),$$

ou le '+' est la concaténation. Écrire une fonction `thue_morse` qui prend en entrée un entier n positif et qui renvoie la suite de Thue-Morse après n étapes.

Exemples :

```
>>> thue_morse(0)
'0'
>>> thue_morse(1)
'01'
>>> thue_morse(2)
'0110'
```

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes:

Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet:

On construit une suite de listes selon la règle suivante :

$$L_0 = [1], \quad \text{et pour tout } n \geq 1, \quad L_n = L_{n-1} + [n+1] + L_{n-1},$$

ou l'opération $+$ représente la concaténation. Autrement dit, à chaque étape, on ajoute un entier au centre de la liste et on recopie la liste précédente de chaque côté.

Exemple de construction :

$$L_0 = [1], \quad L_1 = [1, 2, 1], \quad L_2 = [1, 2, 1, 3, 1, 2, 1].$$

1. Écrire une fonction `etape_miroir` qui prend en entrée une liste L et un entier et renvoie une nouvelle liste construite selon la règle du miroir : `nouvelle_liste = L + [n] + L`.
2. Écrire une fonction `suite_miroir` qui prend en entrée un entier n et qui envoie la liste L_n construite selon la règle du miroir.

Remarques étudiant :

Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: Le jeu **Wordle** consiste à deviner un mot choisi aléatoirement parmi une liste de mots. Le joueur propose des mots et reçoit des indications sur les lettres correctes et leur position. Nous allons simuler la partie "tirage d'un mot mystère" et quelques fonctions utiles. On dispose d'une liste de mots valides :

```
mots = ["APPLE", "BANJO", "CRANE", "DRIVE", "EAGLE"]
```

1. Écrire une fonction `tirer_mot` qui prend en entrée la liste de mots et qui renvoie un mot à deviner, choisi au hasard dans cette liste.

Nous rappelons que pour simuler le tirage aléatoire vous pouvez utiliser la fonction `random.randint(a,b)` qui renvoie un entier aléatoire entre a et b .

Exemple d'usage:

```
import random

print(random.randint(3,5))
```

2. Écrire une fonction `comparer_mots` qui prend en argument le mot à deviner et le mot proposé par le joueur et renvoie une liste de 5 caractères comme suit:
 - 'T' si la lettre est correcte et bien placée (True)
 - 'P' si la lettre est dans le mot mais mal placée (Presque)
 - '-' si la lettre n'est pas dans le mot

Exemple :

```
>>> comparer_mots("CRANE", "CARTE")
['T', 'P', 'P', '-', 'T']
```

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes:

Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet:

Vous préparez vos vacances et souhaitez visiter plusieurs lieux touristiques, tout en limitant votre impact écologique. Chaque lieu est situé à une certaine distance de votre point de départ. Il y a 9 lieux possibles, dont la distance depuis votre point de départ est stockée dans la liste `liste_distances` (en kilomètres). Vous avez défini une distance minimale `dist_min` et une distance maximale `dist_max` pour chaque visite.

1. Écrire une fonction `visites_dans_distance(liste_distances, dist_min, dist_max)` qui renvoie la liste des lieux dont la distance est comprise entre `dist_min` et `dist_max`.
2. Votre voisin souhaite choisir le lieu aléatoirement parmi les lieux de la liste, mais que la distance soit comprise entre `dist_min` et `dist_max`. Écrire une fonction `lieu_aleatoire(liste_distances)` qui choisit sa destination de manière aléatoire.

Vous pourrez tester vos fonctions avec la liste suivante :

```
liste_distances = [12.0, 25.0, 8.0, 10.0, 12.5, 15.0, 5.5, 22.5, 18.8]
```

On rappelle que la fonction `random.randint(a,b)` renvoie un entier aléatoire entre *a* et *b*.

```
import random
print(random.randint(3,9))
```

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes:

Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet:

Un tournoi amical est organisé entre 23 joueurs, numérotés de 0 à 22. Chaque joueur doit affronter un seul adversaire, et personne ne doit se retrouver contre lui-même. Le tirage au sort des matchs est représenté par une liste **rencontres** contenant exactement une fois chaque entier de 0 à 22.

Ainsi, si `rencontres[i] = j`, cela signifie que le joueur *i* affrontera le joueur *j*.

1. Écrire une fonction `verifie_tirage(rencontres)` qui prend en argument une liste d'entiers représentant un tirage des rencontres, et qui vérifie qu'**aucun joueur ne joue contre lui-même**,. La fonction doit retourner `True` si le tirage est valide (aucun joueur contre lui-même), et `False` sinon.
2. Écrire une fonction `tirage_aleatoire()` qui crée une liste contenant les entiers de 0 à 22 et qui mélange cette liste jusqu'à obtenir une liste de rencontres valide et qui la renvoie. Vous pourrez utiliser la fonction `random.shuffle()` permet de mélanger les éléments d'une liste.

Exemple d'usage:

```
import random

liste=[12,11,3]
random.shuffle(liste)
print(liste)
```

Remarques étudiant :
Remarques correcteur :